第三章　　栈和队列

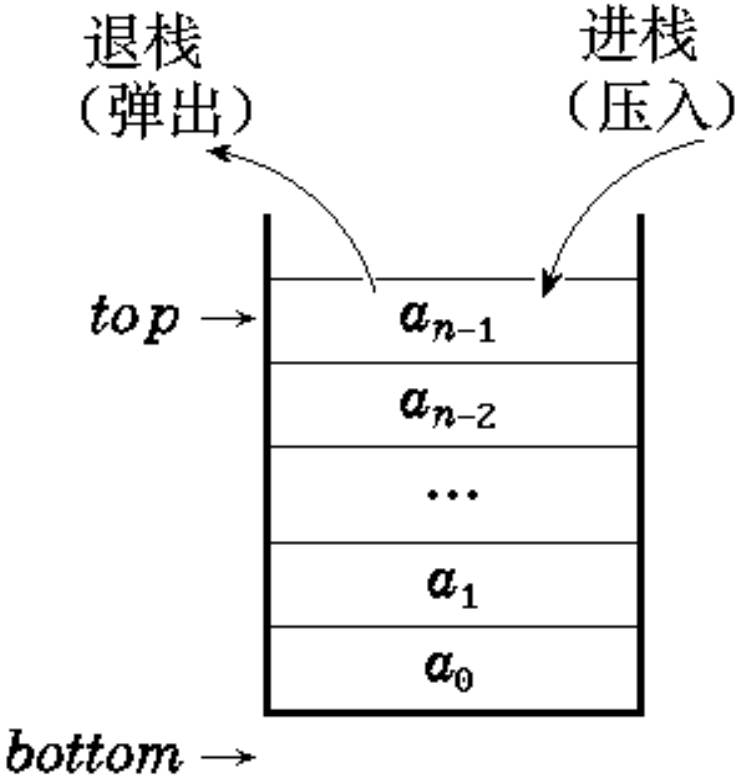
栈
队列
优先级队列
} 线性结构

与表不同的是，它们都是限制存取位置的线性结构

§3.1 栈（stack）

- 只允许在一端插入和删除的线性表
- 允许插入和删除的一端称为栈顶(*top*)，另一端称为栈底(*bottom*)
- 特点

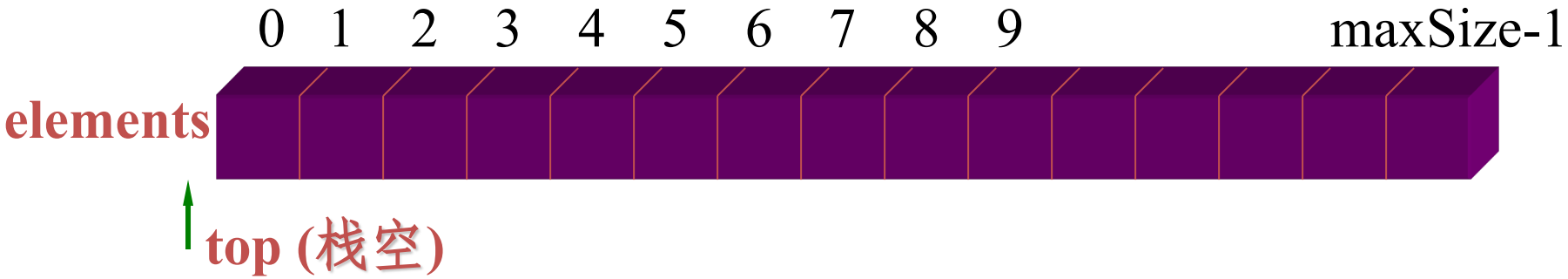
  后进先出(*LIFO*)



栈的抽象数据类型

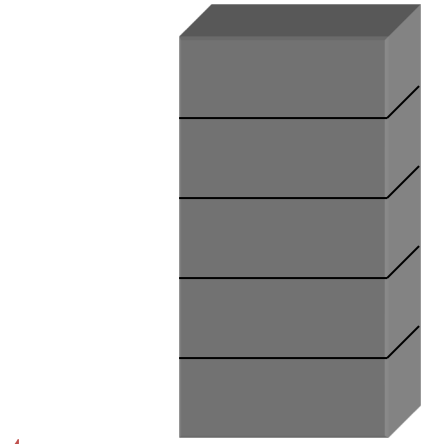
```
template <class E> class Stack {
public:
    Stack ( int=10 );                    //构造函数
    void Push ( const E & item);  //进栈
    E Pop ( );                           //出栈
    E getTop ( );                        //取栈顶元素
    void makeEmpty ( );                  //置空栈
    int IsEmpty ( ) const;               //判栈空否
    int IsFull ( ) const;                //判栈满否
}
```

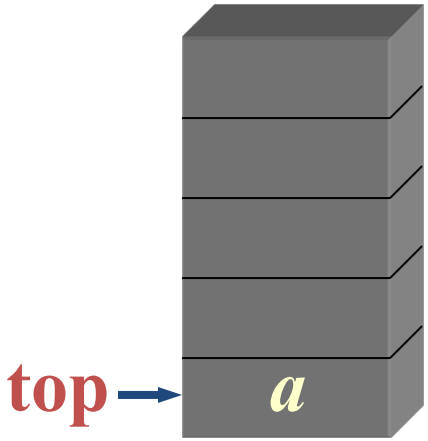
栈的数组表示 — 顺序栈

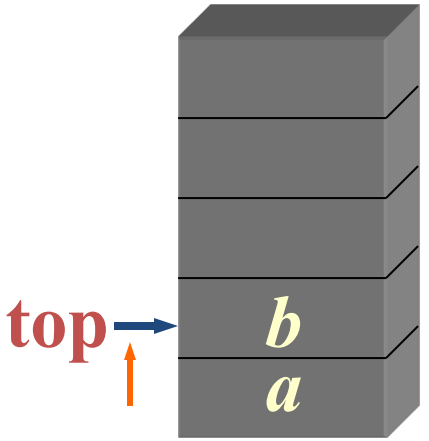


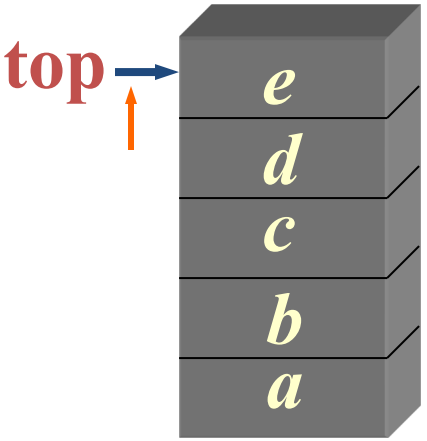

```
#include <assert.h>
template <class E>
class SeqStack : public Stack<E> {
private:
    int top;                        //栈顶指针
    E *elements;                    //栈元素数组
    int maxSize;                    //栈最大容量
    void overflowProcess();         //栈的溢出处理
```

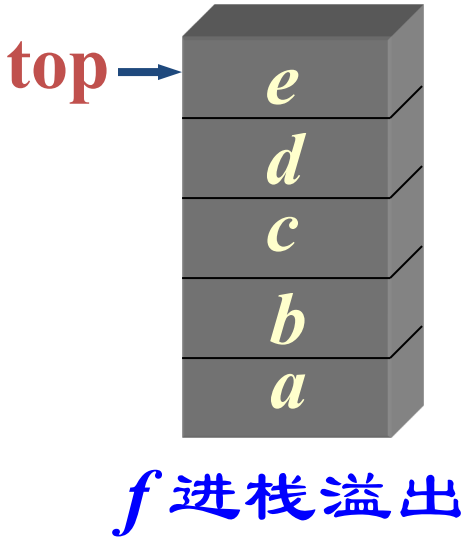
```
public:
    Stack (int sz = 10);                    //构造函数
    ~Stack ( ) { delete [ ] elements; }
    void Push (E x);                        //进栈
    int Pop (E& x);                         //出栈
    int getTop (E& x);                      //取栈顶
    void makeEmpty ( ) { top = -1; }        //置空栈
    int IsEmpty ( ) const { return top == -1; }
    int IsFull ( ) const
        { return top == maxSize-1; }
};
```



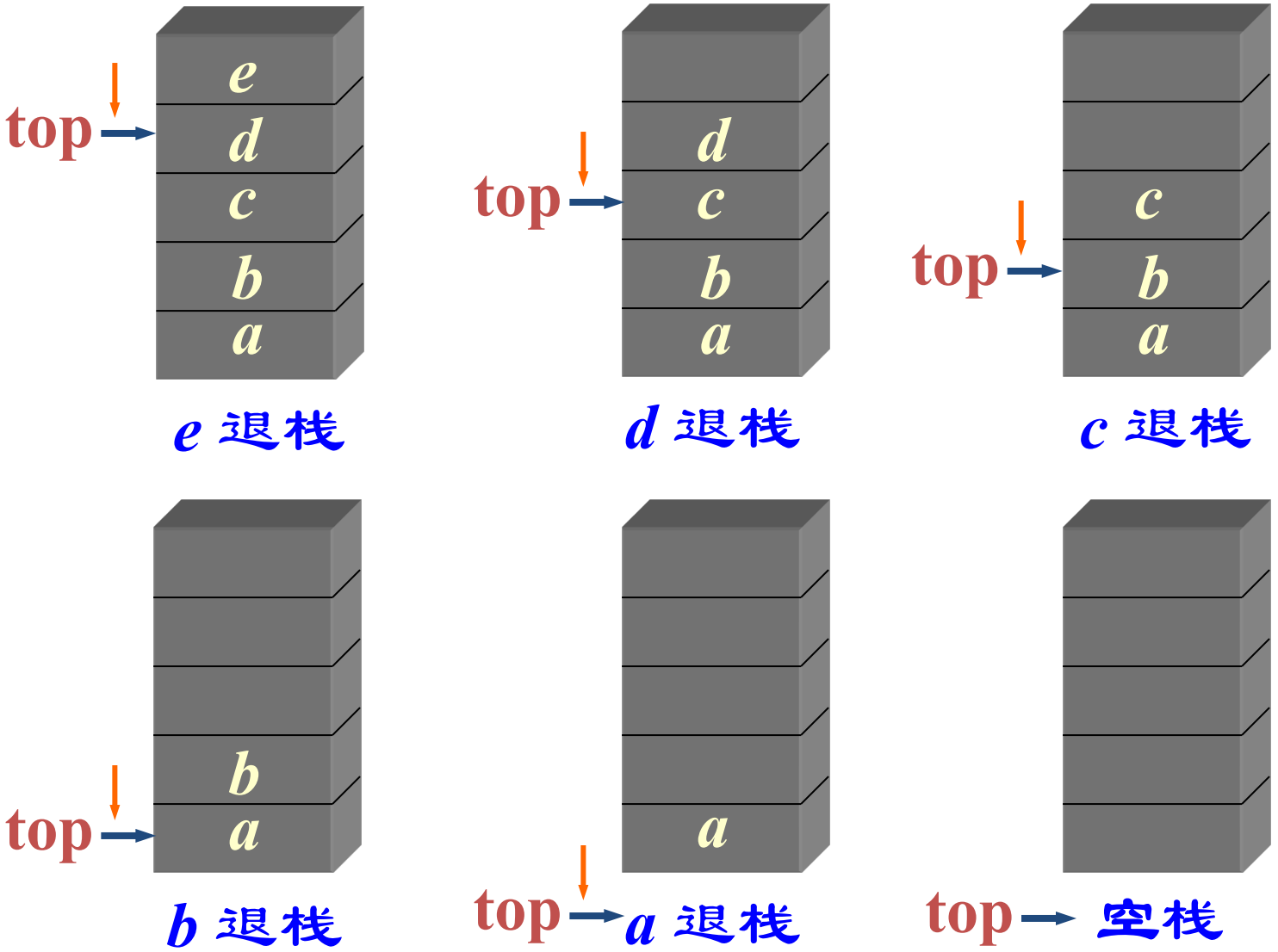








进栈示例


top
e
d
c
b
a
e 退栈
top
d
c
b
a
d 退栈
top
c
b
a
c 退栈
top
b
a
b 退栈
top
a
a 退栈
top
空栈

退栈示例

顺序栈的操作

```
template <class E>
void SeqStack<E>::overflowProcess() {
//私有函数：当栈满则执行扩充栈存储空间处理
    E *newArray = new E[2*maxSize];
                //创建更大的存储数组
    for (int i = 0; i <= top; i++)
        newArray[i] = elements[i];
    maxSize += maxSize;
    delete [ ]elements;
    elements = newArray;    //改变elements指针
};
```

```
template <class E>
void SeqStack<E>::Push(E x) {
//若栈不满,则将元素x插入该栈栈顶,否则溢出处理
    if (IsFull() == true) overflowProcess();        //栈满
    elements[++top] = x;      //栈顶指针先加1,再进栈
};

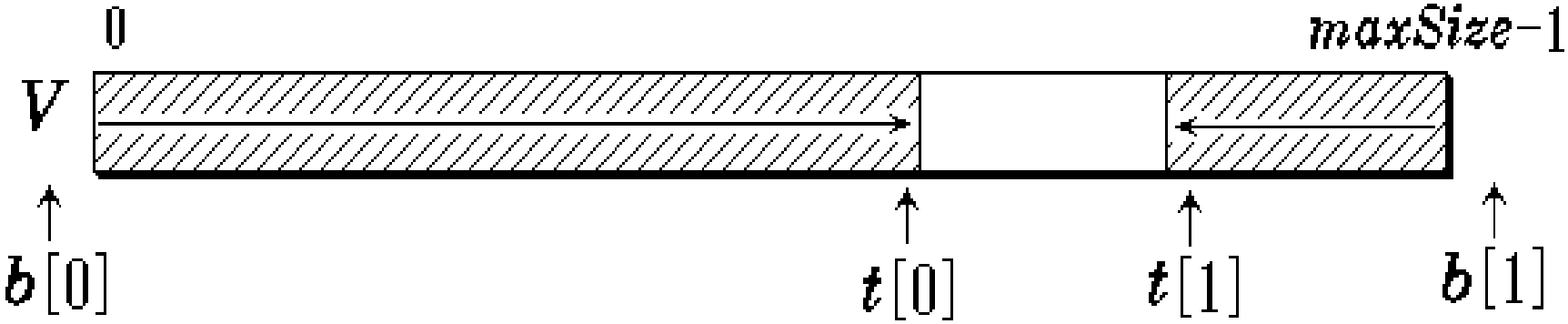
template <class E>
bool SeqStack<E>::Pop(E& x) {
//函数退出栈顶元素并返回栈顶元素的值
    if (IsEmpty() == true) return false;
    x = elements[top--];      //栈顶指针退1
    return true;              //退栈成功
};
```

```
template <class E>
bool Seqstack<E>::getTop(E& x) {
//若栈不空则函数返回该栈栈顶元素的地址
    if (IsEmpty() == true) return false;
    x = elements[top];
    return true;
};
```

如何合理进行栈空间分配，以避免栈溢出或空间的浪费？

- 双栈共享一个栈空间（多栈共享栈空间）
- 栈的链接存储方式—— 链式栈

双栈共享一个栈空间

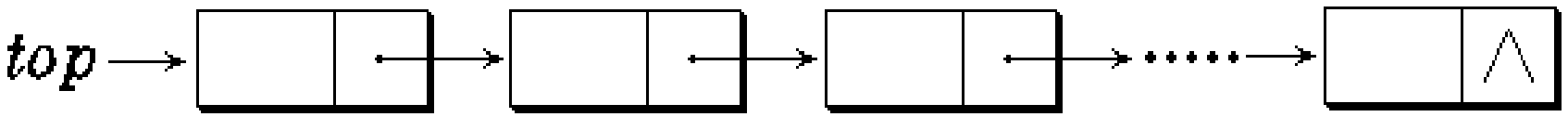


- 两个栈共享一个数组空间V[maxSize]
- 设立栈顶指针数组 t[2] 和栈底指针数组 b[2]
  t[i]和b[i]分别指示第 i 个栈的栈顶与栈底
- 初始 t[0] = b[0] = −1, t[1] = b[1] = maxSize
- 栈满条件: t[0]+1 == t[1] //栈顶指针相遇
- 栈空条件: t[0] = b[0]或t[1] = b[1]
  //栈顶指针退到栈底

```
bool Push(DualStack& DS, E x, int i) {
    if (DS.t[0]+1 == DS.t[1]) return false;
    if (i == 0) DS.t[0]++; else DS.t[1]--;
    DS.V[DS.t[i]] = x;
    return true;
}

bool Pop(DualStack& DS, E & x, int i) {
    if (DS.t[i] == DS.b[i]) return false;
    x = DS.V[DS.t[i]];
    if (i == 0) DS.t[0]--; else DS.t[1]++;
    return true;
}
```

栈的链接表示 — 链式栈



- 链式栈无栈满问题，空间可扩充
- 插入与删除仅在栈顶处执行
- 链式栈的栈顶在链头

链式栈 (LinkedStack) 类的定义

```
#include <iostream.h>
#include "stack.h"
template <class E>
struct StackNode {                    //栈结点类定义
private:
    E data;                           //栈结点数据
    StackNode<E> *link;               //结点链指针
public:
    StackNode(E d = 0, StackNode<E> *next = NULL)
        : data(d), link(next) { }
};
```

```
template <class E>
class LinkedStack : public Stack<E> { //链式栈类定义
private:
    StackNode<E> *top;                    //栈顶指针

public:
    LinkedStack() : top(NULL) {}          //构造函数
    ～LinkedStack() { makeEmpty(); }      //析构函数
    void Push(E x);                       //进栈
    bool Pop(E& x);                       //退栈
```

```
    bool getTop(E& x) const;              //取栈顶
    bool IsEmpty() const { return top == NULL; }
    void makeEmpty();                     //清空栈的内容
     friend ostream& operator << (ostream& os,
        LinkedStack<E> & s) ;
     //输出栈元素的重载操作 <<
};
```

链式栈类操作的实现

```
template <class E>
LinkedStack<E>::makeEmpty() {
//逐次删去链式栈中的元素直至栈顶指针为空。
    StackNode<E> *p;
    while (top != NULL)              //逐个结点释放
       { p = top;  top = top->link;  delete p; }
};

template <class E>
void LinkedStack<E>::Push(E x) {
//将元素值x插入到链式栈的栈顶,即链头
 top = new StackNode<E> (x, top);     //创建新结点
     assert (top != NULL);                    //创建失败退出
};
```

```
template <class E>
bool LinkedStack<E>::Pop(E& x) {
//删除栈顶结点, 返回被删栈顶元素的值。
    if (IsEmpty() == true) return false;   //栈空返回
     StackNode<E> *p = top;                //暂存栈顶元素
     top = top->link;                      //退栈顶指针
     x = p->data;
    delete p;              //释放结点
    return true;
};
```

```
template <class E>
bool LinkedStack<E>::getTop(E& x) const {
     if (IsEmpty() == true) return false;   //栈空返回
     x = top->data;                         //返回栈顶元素的值
    return true;
   };
```

```
template <class E>
ostream& operator << (ostream& os,
        LinkedStack<E> & s) {
    //输出栈中元素的重载操作<<
    os << "栈中元素个数="<<s.getSize() << endl;
    LinkNode<E> * p = s.top; int i = 0;
    while (p != NULL) {
        os << ++i << ":" <<p->data << endl;
         p = p-> link;
     }
     return os;
};
```

思考：当进栈元素的编号为1, 2, …, n时，可能的出栈序列有多少种？

栈的应用：表达式的计算

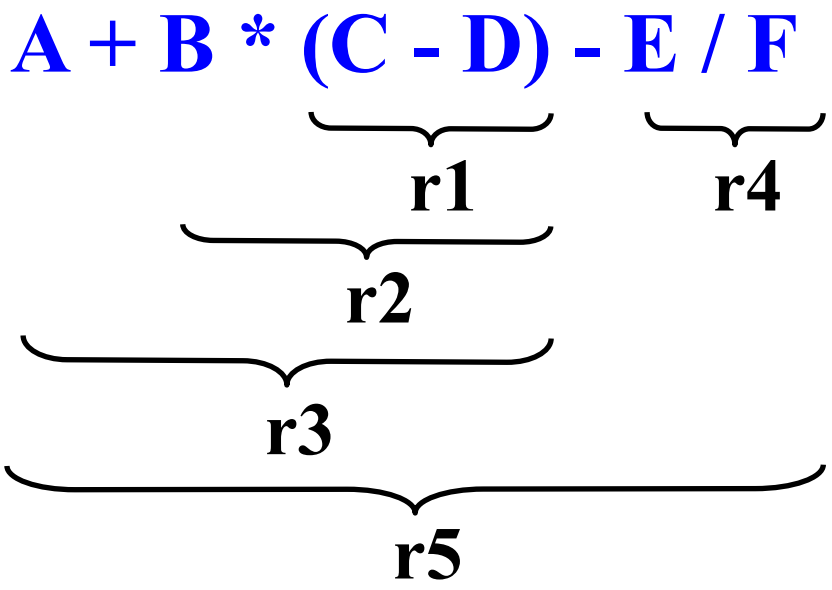
- 算术表达式有三种表示：
  - 中缀(infix)表示

    <操作数> <操作符> <操作数>，如 A+B;
  - 前缀(prefix)表示

    <操作符> <操作数> <操作数>，如 +AB;
  - 后缀(postfix)表示

    <操作数> <操作数> <操作符>，如 AB+;

表达式示例



中缀表达式 → 后缀表达式

A + B * (C - D) - E / F　　ABCD-*+EF/-

中缀表达式　A+B*(C-D)-E/F

- 表达式中相邻两个操作符的计算次序为：
  - 优先级高的先计算
  - 优先级相同的自左向右计算
  - 当使用括号时从最内层括号开始计算

后缀表达式　A B C D - * + E F / -

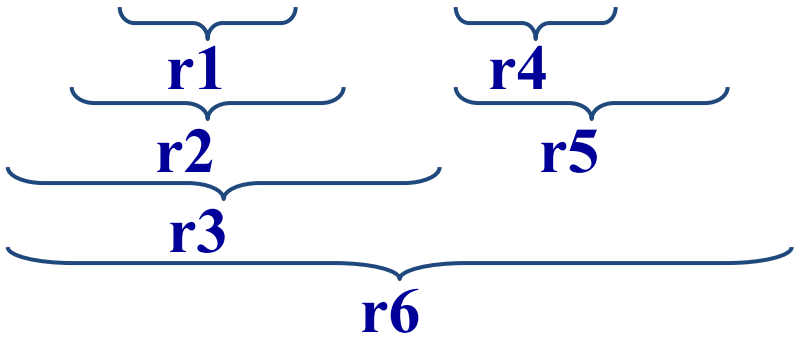
- 在后缀表达式的计算顺序中已隐含了加括号的优先次序，括号在后缀表达式中不出现。

应用后缀表示计算表达式的值

idea:

- 从左向右顺序地扫描表达式，并用一个栈暂存扫描到的操作数或计算结果。
- 扫描中遇操作数则压栈；遇操作符则从栈中退出两个操作数，计算后将结果压入栈
- 最后计算结果在栈顶

- 计算例　A B C D − * + E F ^ G / −



计算后缀表达式 A B C D − * + E F ^ G / −

步	输入	类 型	动 作	栈内容
1			置空栈	空
2	A	操作数	进栈	A
3	B	操作数	进栈	AB
4	C	操作数	进栈	ABC
5	D	操作数	进栈	ABCD
6	−	操作符	D、C 退栈，计算 C−D，结果 r1 进栈	ABr1
7	*	操作符	r1、B 退栈，计算 B*r1，结果 r2 进栈	Ar2
8	+	操作符	r2、A 退栈，计算 A+r2，结果 r3 进栈	r3

计算后缀表达式 AB C D − * + E F ^ G / −

步	输入	类 型	动 作	栈内容
9	E	操作数	进栈	r3E
10	F	操作数	进栈	r3EF
11	^	操作符	F、E 退栈，计算 E^F，结果 r4 进栈	r3r4
12	G	操作数	进栈	r3r4G
13	/	操作符	G、r4 退栈，计算 r4/G，结果 r5 进栈	r3r5
14	−	操作符	r5、r3 退栈，计算 r3−r5，结果 r6 进栈	r6

```
void Calculator :: Run ( ) {
   char ch;   double newoperand;
   while ( cin >> ch,  ch != ';' ) {
      switch ( ch ) {
        case '+' :  case '-' :   case '*' :   case '/' :
         case '^' :  DoOperator ( ch );  break;
                                        //计算
        default :  cin.putback ( ch );
                              //将字符放回输入流
                        cin >> newoperand;  //读操作数
                        S.Push( newoperand );
      }
   }
}
```

```
void Calculator :: DoOperator ( char op ) {
//从栈S中取两个操作数，形成运算指令并计算进栈
   double left, right;   bool result;
   result = Get2Operands(left, right); //退出两个操作数
   if ( !result ) return;
   switch ( op ) {
      case '+': S.Push ( left + right);  break;     //加
      case '-': S.Push ( left -  right);  break;     //减
      case '*': S.Push ( left * right);  break;     //乘
      case '/': if ( right != 0.0 ) {S.Push ( left / right); break; }
                else { cout << "除数为0!\n" );  exit(1); } //除
      case '^': S.Push ( Power(left,right) ); //乘幂
   }
}
```

```
bool Calculator :: Get2Operands(double& left,
                                double& right ) {
//从栈S中取两个操作数
    if (S.IsEmpty( ) == true)
        {cerr << "缺少右操作数！ "<< endl; return false;}
    S.Pop(right);

    if (S.IsEmpty( ) == true)
        {cerr << "缺少左操作数！ "<< endl; return false;}
    S.Pop(left);

    return true;
}
```

- 一般表达式的操作符有4种类型:

1 算术操作符 如双目操作符（+、-、*、/和%）以及单目操作符（-）。

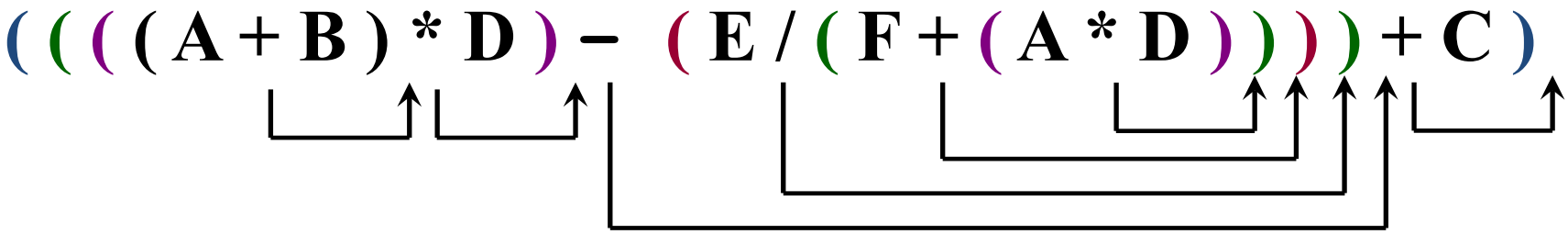
2 关系操作符 包括<、<=、==、!=、>=、>。这些操作符主要用于比较。

3 逻辑操作符 如与(&&)、或(||)、非(!)。

4 括号 ‘(’和 ‘)’，它们的作用是改变运算顺序。

中缀表示→ 后缀表示

- 先对中缀表达式按运算优先次序加上括号，再把操作符后移到右括号的后面并以就近移动为原则，最后将所有括号消去。
- 如中缀表示 (A+B)*D-E/(F+A*D)+C，其转换为后缀表达式的过程如下:



后缀表示 A B + D * E F A D * + / – C +

利用栈将中缀表示转换为后缀表示

- 使用栈可将表达式的中缀表示转换成它的后缀表示。

- 为了实现这种转换，需要考虑各操作符的优先级。

各个算术操作符的优先级

操作符 ch	;	(	*, /, %	+, −	)
isp (栈内)	0	1	5	3	6
icp (栈外)	0	6	4	2	1

- isp叫做栈内(in stack priority)优先级
- icp叫做栈外(in coming priority)优先级。
- 操作符优先级相等的情况只出现在括号配对或栈底的“;”号与输入流最后的“;”号配对时。

中缀表达式转换为后缀表达式的算法

- 操作符栈初始化，将结束符 ‘;’进栈。然后读入中缀表达式字符流的首字符ch。
- 重复执行以下步骤，直到ch = ‘;’，同时栈顶的操作符也是 ‘;’，停止循环。
  - 若ch是操作数直接输出，读入下一个字符ch。
  - 若ch是操作符，判断ch的优先级icp和位于栈顶的操作符op的优先级isp:

- 若 icp(ch) > isp(op)，令ch进栈，读入下一个字符ch。
- 若 icp(ch) < isp(op)，退栈并输出。
- 若 icp(ch) == isp(op)，退栈但不输出，若退出的是“(”号读入下一个字符ch。

算法结束，输出序列即为所需的后缀表达式

步	输入	栈内容	语义	输出	动作
1		;			栈初始化
2	A	;		A	操作数A输出，读字符
3	+	;	+ > ;		操作符+进栈，读字符
4	B	;+		B	操作数B输出，读字符
5	*	;+	* > +		操作符*进栈，读字符
6	(	;+*	(> *		操作符(进栈，读字符
7	C	; +*(		C	操作数C输出，读字符
8	–	; +*(	– > (		操作符–进栈，读字符
9	D	; +*(–		D	操作数D输出，读字符
10	)	; +*(–	) < –	–	操作符–退栈输出
11		; +*(	) = (		(退栈，消括号，读字符

步	输入	栈内容	语义	输出	动作
12	–	;+*	– < *	*	操作符*退栈输出
13		;+	– < +	+	操作符+退栈输出
14		;	– > ;		操作符–进栈，读字符
15	E	;–		E	操作数E输出，读字符
16	/	;–	/ > –		操作符/进栈，读字符
17	F	;–/		F	操作数F输出，读字符
18	;	;–/	; < /	/	操作符/退栈输出
19		;–	; < –	–	操作符–退栈输出
20		;	; = ;		;配对，转换结束

§3.2 栈与递归

- 递归的定义

  若一个对象部分地包含它自己，或用它自己给自己定义，则称这个对象是递归的；若一个过程直接地或间接地调用自己，则称这个过程是递归的过程。

- 以下三种情况常常用到递归方法。
  - 定义是递归的
  - 数据结构是递归的
  - 问题的解法是递归的

定义是递归的

例如，阶乘函数

$$n!=\begin{cases}1, & \text{当 } n=0 \text{ 时}\\ n*(n-1)!, & \text{当 } n\geq 1 \text{ 时}\end{cases}$$

求解阶乘函数的递归算法

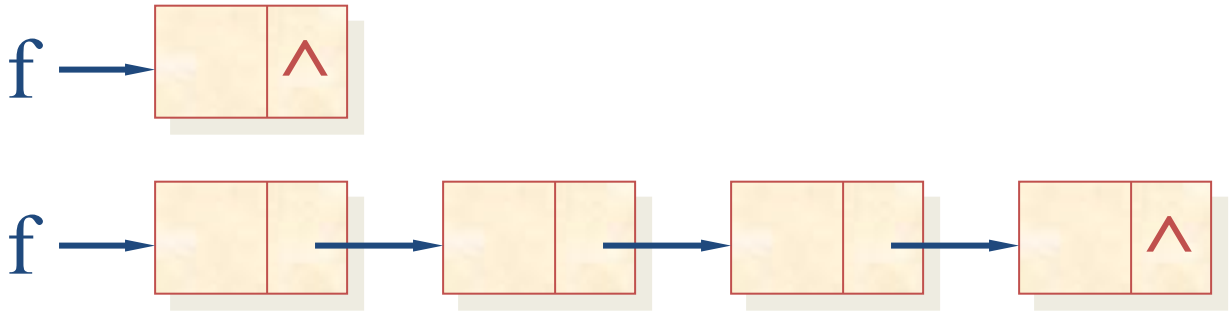
```
long Factorial(long n) {
    if (n == 0) return 1;
    else return n*Factorial(n−1);
}
```

求解阶乘 $n!$ 的过程


主程序 main : factorial(4)
参数 4　计算 4*factorial (3)　返回 24
参数 3　计算 3*factorial (2)　返回 6
参数 2　计算 2*factorial (1)　返回 2
参数 1　计算 1*factorial (0)　返回 1
参数 0　直接定值 = 1　返回 1
递归调用
参数传递
结果返回
返回求值

数据结构是递归的

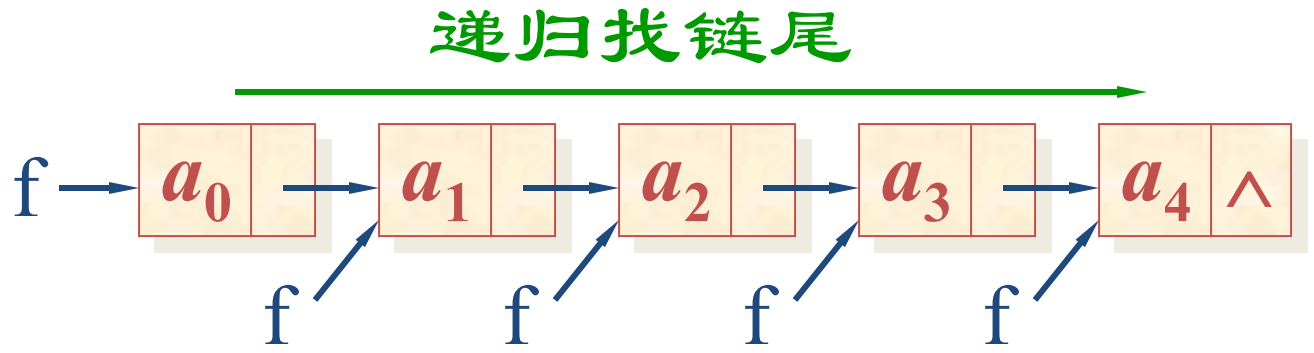
- 例如，单链表结构



- 一个结点，它的指针域为NULL，是一个单链表；
- 一个结点，它的指针域指向单链表，仍是一个单链表。

搜索链表最后一个结点并打印其数值

```
template <class E>
void Print(ListNode<E> *f) {
    if (f->link == NULL)
        cout << f->data << endl;
    else Print(f->link);
}
```



问题的解法是递归的

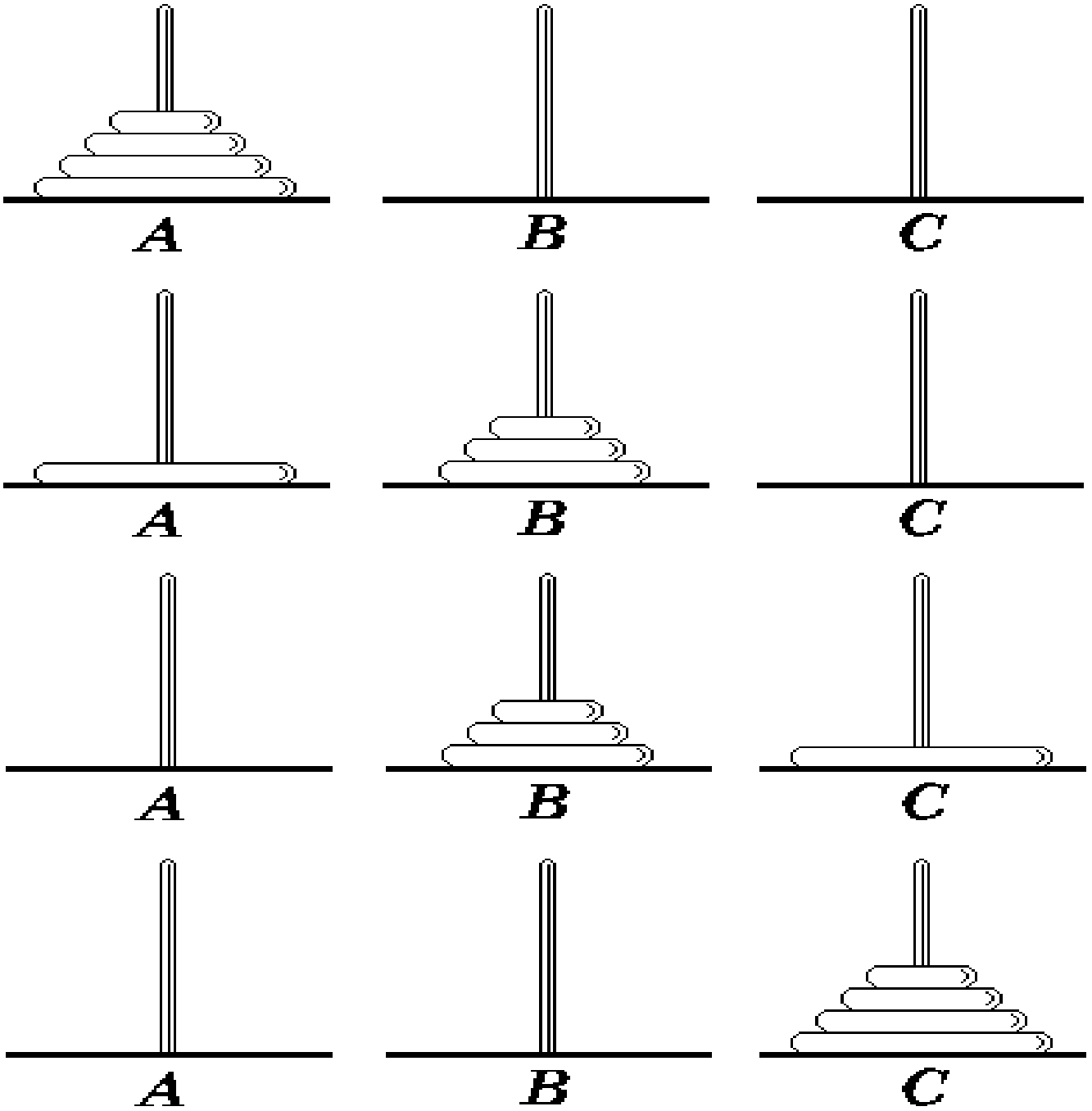
- 例，汉诺塔(Tower of Hanoi)问题的解法:

  如果 n = 1，则将这一个盘子直接从 A 柱移到 C 柱上。否则，执行以下三步:

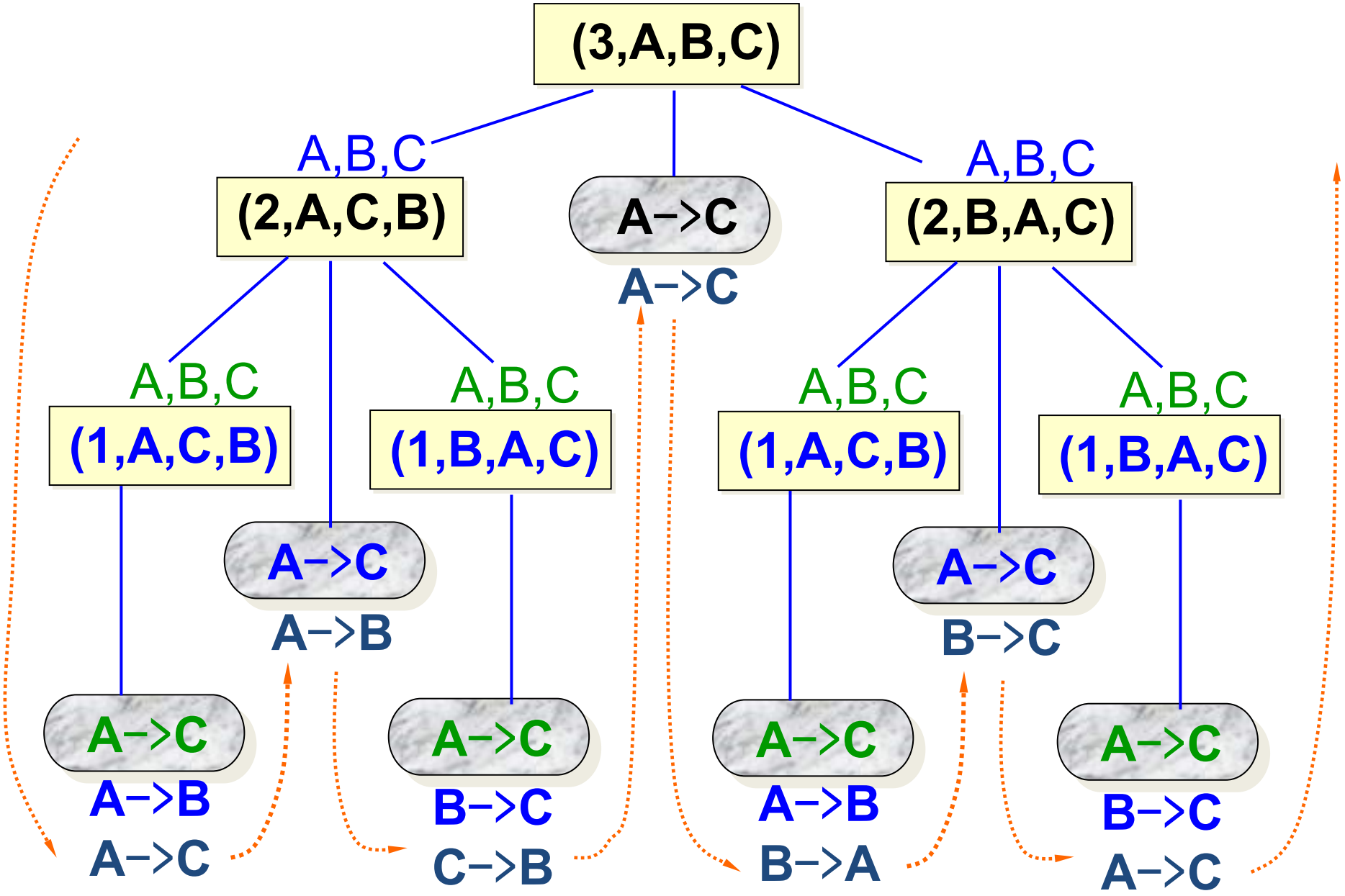
  ① 用 C 柱做过渡，将 A 柱上的 (n-1) 个盘子移到 B 柱上:

  ② 将 A 柱上最后一个盘子直接移到 C 柱上;

  ③ 用 A 柱做过渡，将 B 柱上的 (n-1) 个盘子移到 C 柱上。


A
B
C
A
B
C
A
B
C
A
B
C

```
#include <iostream.h>
void Hanoi (int n, char A, char B, char C) {
//解决汉诺塔问题的算法
    if (n == 1) cout << " move " << A << " to  "
          << C << endl;
    else {  Hanoi(n-1, A, C, B);
            cout << " move " << A << " to " << C
                  << endl;
            Hanoi(n-1, B, A, C);
          }
}
```


(3,A,B,C)
A,B,C
(2,A,C,B)
A–>C
A–>C
A,B,C
(2,B,A,C)
A,B,C
(1,A,C,B)
A,B,C
(1,B,A,C)
A,B,C
(1,A,C,B)
A,B,C
(1,B,A,C)
A–>C
A–>B
A–>C
B–>C
A–>C
A–>B
A–>C
B–>C
A–>C
A–>B
A–>C
B–>C
C–>B
B–>A
A–>C

什么时候运用递归？

- 子问题应与原问题做同样的事情，且更为简单；
- 把一个规模比较大的问题分解为一个或若干规模比较小的问题，分别对这些比较小的问题求解，再综合它们的结果，从而得到原问题的解。

—分而治之策略（分治法）

- 这些比较小的问题的求解方法与原来问题的求解方法一样。

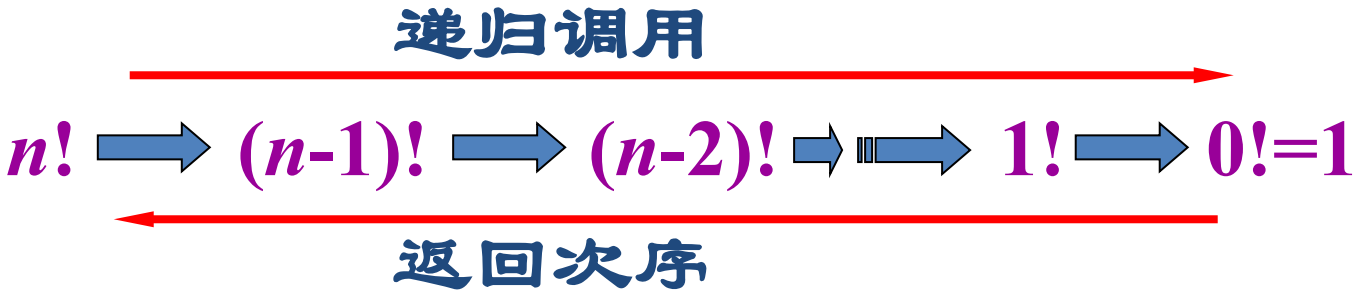
构成递归的条件

- 不能无限制地调用本身，必须有一个出口，化简为非递归状况直接处理。

```
Procedure <name> ( <parameter list> ) {
  if ( < initial condition> )    //递归结束条件
    return ( initial value );
  else                           //递归
    return (<name> ( parameter exchange ));
}
```

递归过程与递归工作栈

- 递归过程在实现时，需要自己调用自己。
- 层层向下递归，退出时的次序正好相反：



- 主程序第一次调用递归过程为外部调用；
- 递归过程每次递归调用自己为内部调用。
- 它们返回调用它的过程的地址不同。

```
long Factorial(long n) {
int temp;
if (n == 0) return 1;
else temp = n * Factorial(n−1);
```

RetLoc2

```
   return temp;
}

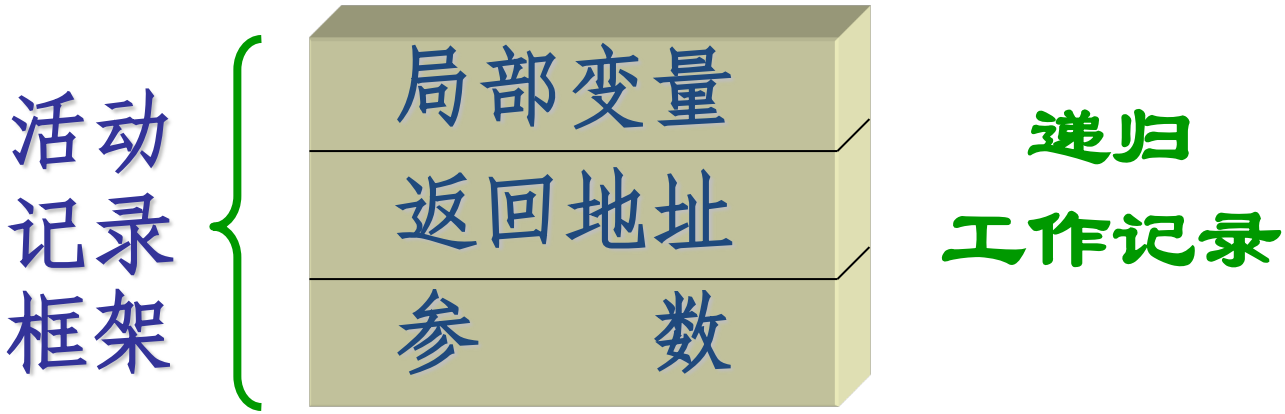
void main() {
   int n;
   n = Factorial(4);
```

RetLoc1

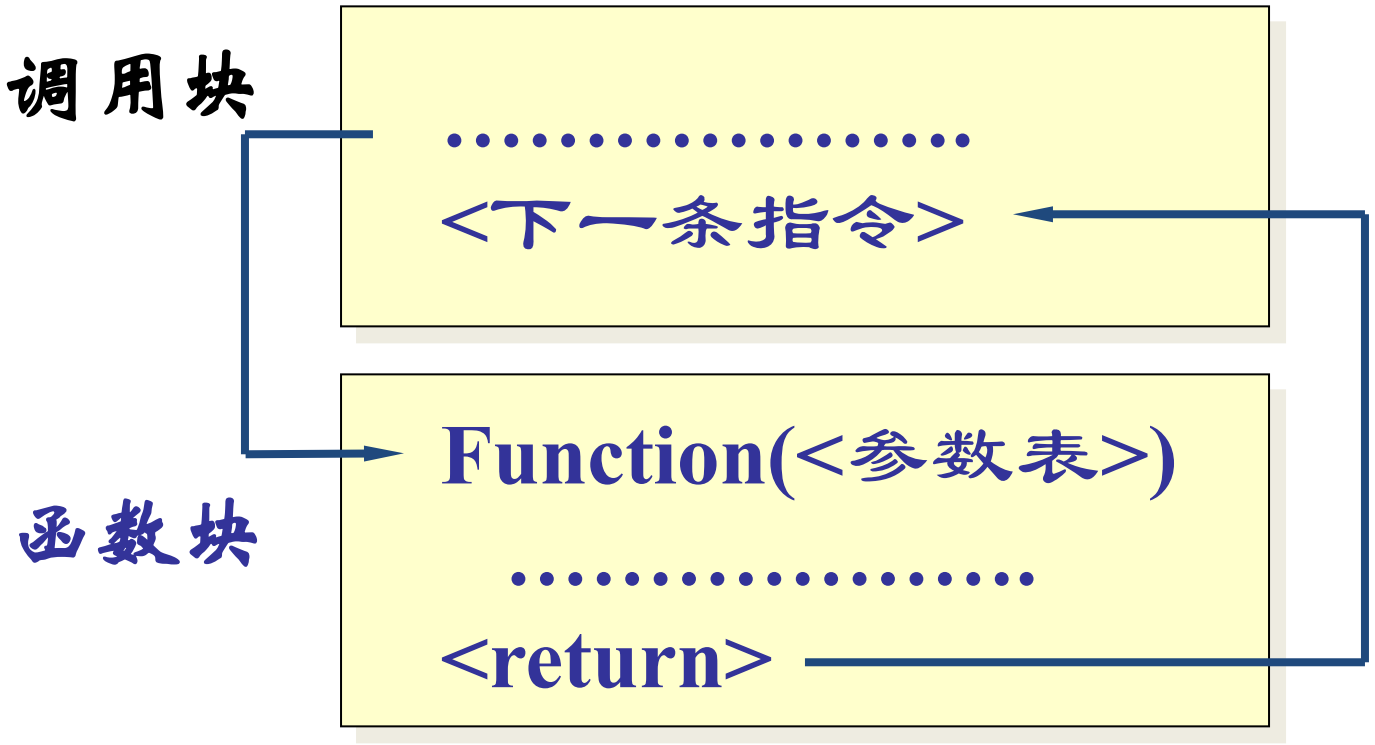
```
}
```

递归工作栈

- 每一次递归调用时，需要为过程中使用的参数、局部变量等另外分配存储空间。
- 每层递归调用需分配的空间形成递归工作记录，按后进先出的栈组织。



递归工作栈



函数递归时的活动记录



计算Factorial时活动记录的内容



递归过程改为非递归过程

- 递归过程简洁、易编、易懂
- 递归过程效率低，重复计算多
- 改为非递归过程的目的是提高效率
- 单向递归和尾递归可直接用迭代实现其非递归过程
- 其他情形必须借助栈实现非递归过程

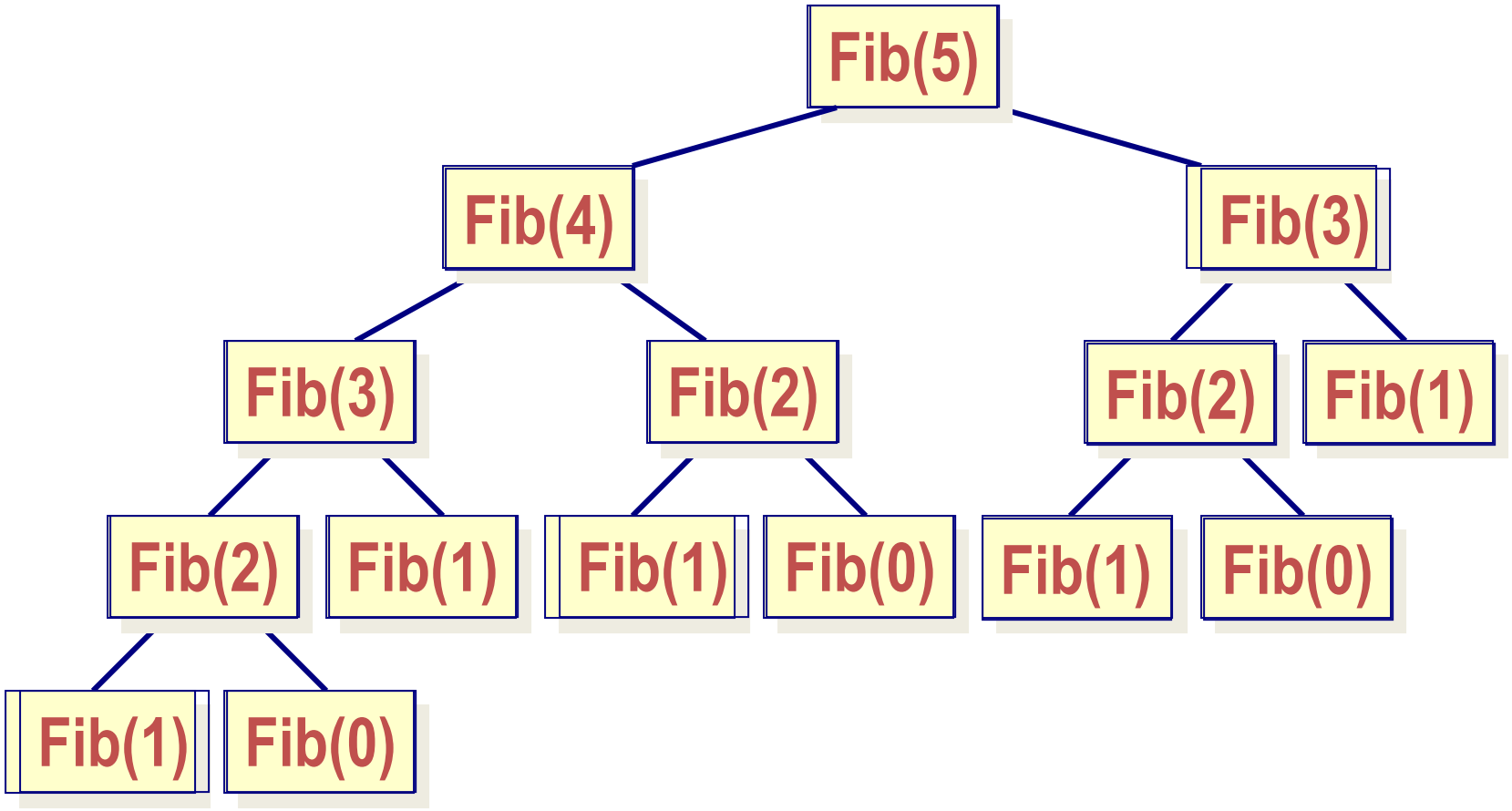
计算斐波那契数列的函数Fib(n)的定义

$$\mathrm{Fib}(n)=\begin{cases} n, & n=0,1 \\ \mathrm{Fib}(n-1)+\mathrm{Fib}(n-2), & n>1 \end{cases}$$

如 $\mathbf{F_0=0, F_1=1, F_2=1, F_3=2, F_4=3, F_5=5}$

求解斐波那契数列的递归算法

```
long Fib(long n) {
    if (n <= 1) return n;
    else return Fib(n-1)+Fib(n-2);
}
```



斐波那契数列的递归调用树

调用次数 **NumCall(k) = 2*Fib(k+1)−1**

单向递归用迭代法实现

```
long FibIter(long n) {
    if (n <= 1) return n;
    long twoback = 0,  oneback = 1,  Current;
    for (int i = 2; i <= n; i++) {
        Current = twoback + oneback;
        twoback = oneback;   oneback = Current;
    }
    return Current;
}
```

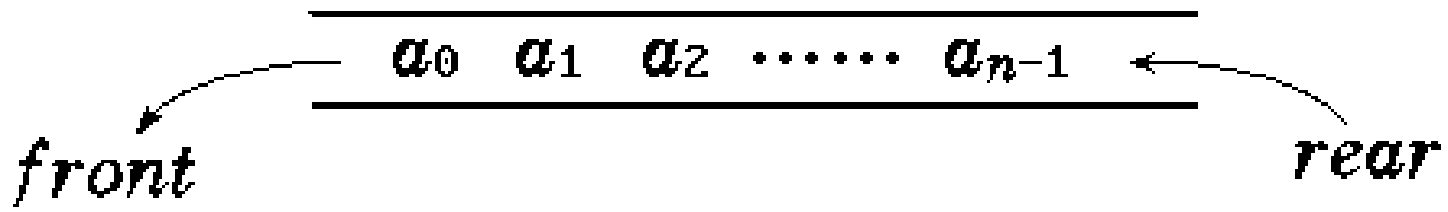
尾递归用迭代法实现

25	36	72	18	99	49	54	63

```
void recfunc(int A[ ],  int n) {
    if (n >= 0) {
        cout << A[n] << “  ”;
        n--;
        recfunc(A, n);
    }
}
```

```
void sterfunc(int A[ ], int n) {
//消除了尾递归的非递归函数
    while (n >= 0) {
        cout << "value    " << A[n] << endl;
        n--;
    }
}
```

§3.3 队列（queue）



- 定义
  - 队列是只允许在一端删除，在另一端插入的线性表
  - 允许删除的一端叫做队头(*front*)，允许插入的一端叫做队尾(*rear*)。
- 特性
  - 先进先出(*FIFO, First In First Out*)

队列的抽象数据类型

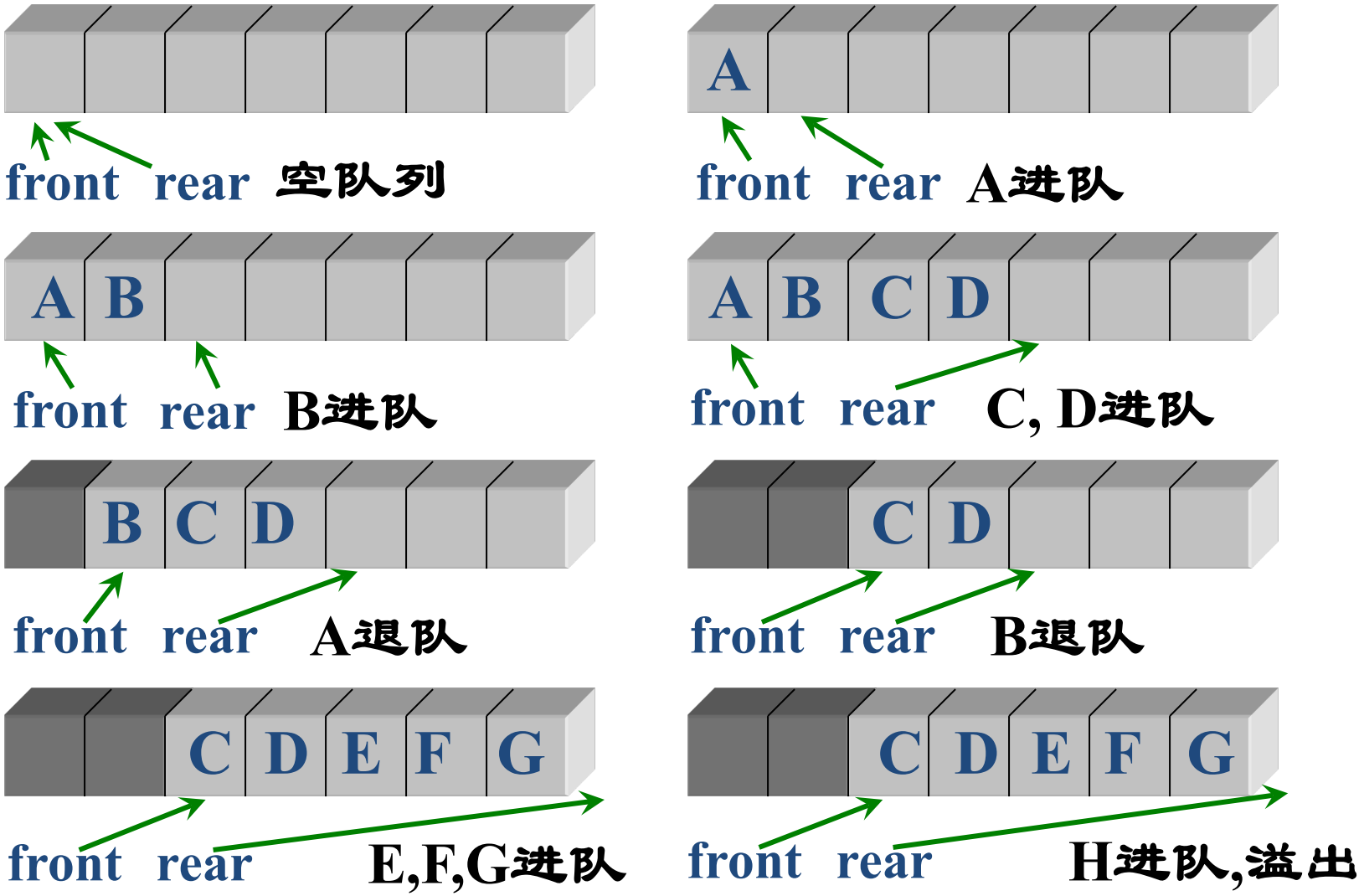
```
template <class E>
class Queue {
public:
    Queue() { };                    //构造函数
    ~Queue() { };                   //析构函数
    virtual bool EnQueue(E x) = 0;          //进队列
    virtual bool DeQueue(E& x) = 0;         //出队列
    virtual bool getFront(E& x) = 0;        //取队头
    virtual bool IsEmpty() const = 0;       //判队列空
    virtual bool IsFull() const = 0;        //判队列满
};
```

队列的数组存储表示 —顺序队列

```
#include <assert.h>
#include <iostream.h>
#include "Queue.h"
template <class E>
class SeqQueue : public Queue<E> {    //队列类定义
protected:
    int rear, front;                  //队尾与队头指针
    E *elements;                      //队列存放数组
    int maxSize;                      //队列最大容量
public:
   SeqQueue(int sz = 10);    //构造函数
```

```
    ~SeqQueue() { delete[ ] elements; }  //析构函数
    bool EnQueue(E x);          //新元素进队列
    bool DeQueue(E& x);         //退出队头元素
    bool getFront(E& x);        //取队头元素值
    void makeEmpty() { front = rear = 0; }
    bool IsEmpty() const { return front == rear; }
    bool IsFull() const
        { return rear ==maxSize; }
    int getSize() const
        { return rear-front; }
};
```

队列的进队和出队（数组方式）

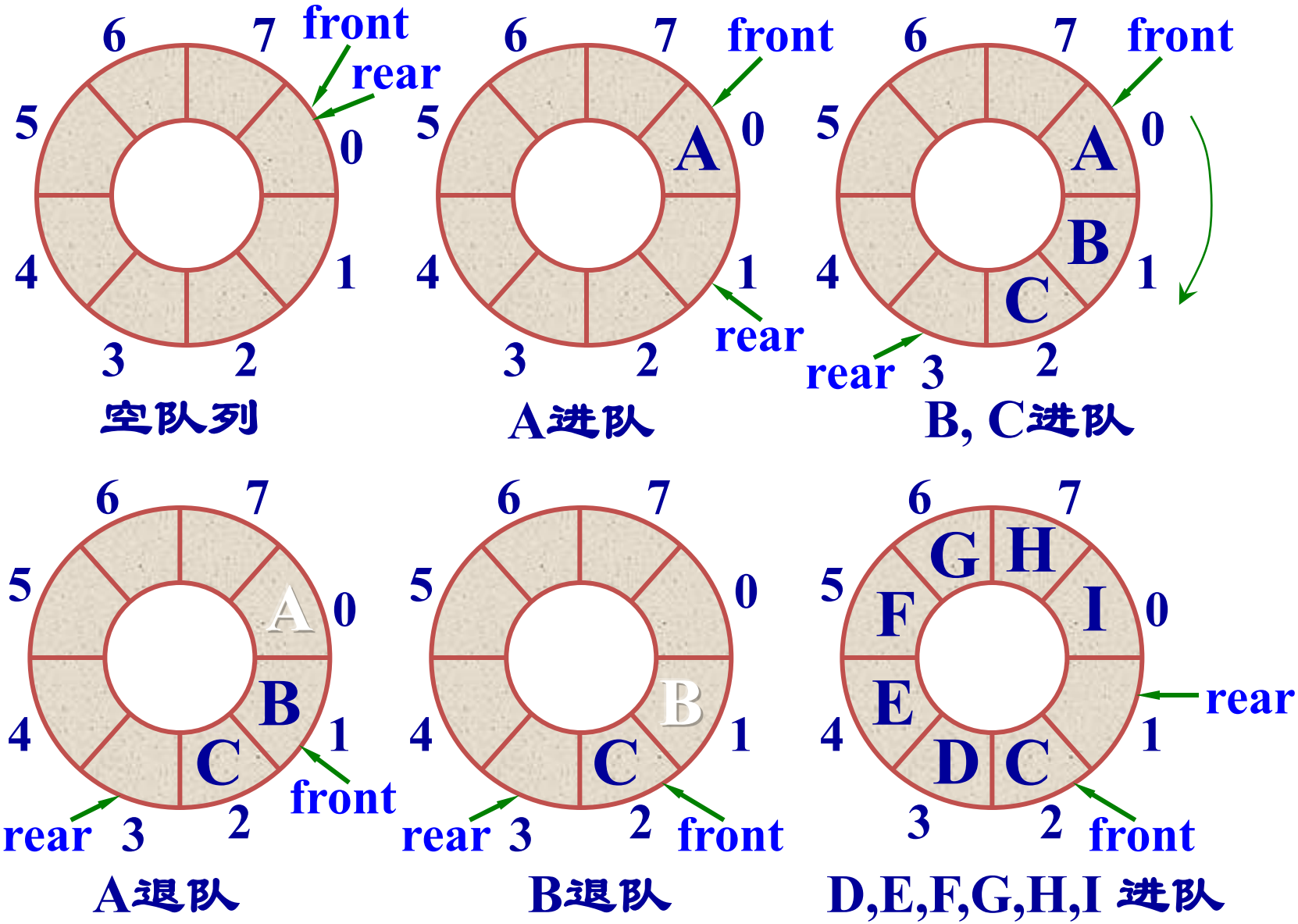


队列的进队和出队

- 进队:新元素在*rear*处加入，*rear* = *rear* + 1。
- 出队:取出下标为 *front* 的元素，*front* = *front* + 1
- 队空时 *rear* == *front*
- 队满时 *rear* == *maxSize*（假溢出）
- 解决假溢出的办法之一：将队列元素存放数组首尾相接，形成循环（环形）队列。

循环队列 (Circular Queue)

- 队列存放数组被当作首尾相连的表处理。
- 队头、队尾指针加1时从**maxSize-1**直接进到**0**, 可用语言的取模(余数)运算实现。
- 队头指针进**1**: front = (front+1) % maxSize;
- 队尾指针进**1**: rear = (rear+1) % maxSize;
- 队列初始化: front = rear = 0;
- 队空条件: front == rear;
- 队满条件: (rear+1) % maxSize == front


6
7
front
rear
5
0
4
1
3
2
空队列
A
A进队
B
C
rear
B, C进队
A退队
B退队
G
H
F
I
E
D
rear
front
D,E,F,G,H,I 进队

循环队列操作的定义

```
void MakeEmpty() { front = rear = 0; }
int IsEmpty() const { return front == rear; }
int IsFull() const
    { return (rear+1) % maxSize == front; }

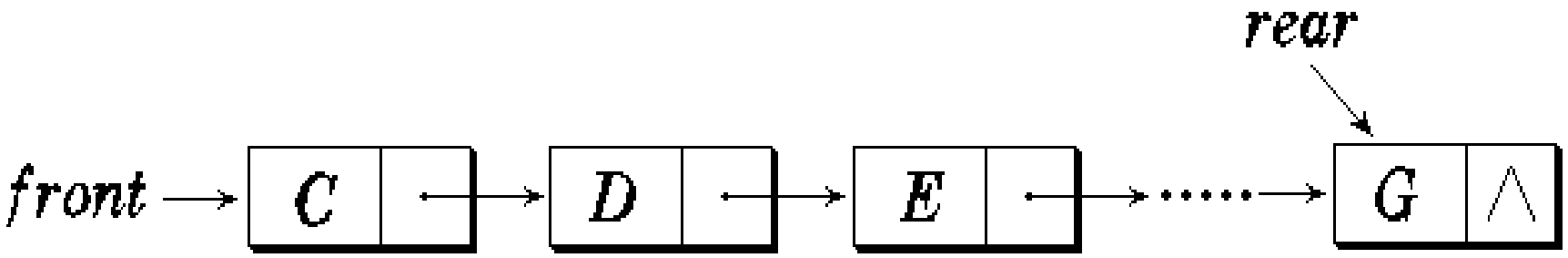
template <class E>
SeqQueue<E>::SeqQueue(int sz)
    : front(0), rear(0), maxSize(sz) {     //构造函数
      elements = new E[maxSize];
      assert ( elements != NULL );
};
```

```
template <class E>
bool SeqQueue<E>::EnQueue(E x) {
//若队列不满, 则将x插入到该队列队尾, 否则返回
  if (IsFull() == true) return false;
     elements[rear] = x;                    //先存入
     rear = (rear+1) % maxSize;             //尾指针加一
     return true;
};
```

```
template <class E>
bool SeqQueue<E>::DeQueue(E& x) {
//若队列不空则函数退队头元素并返回其值
    if (IsEmpty() == true) return false;
    x = elements[front];                  //先取队头
    front = (front+1) % maxSize;   //再队头指针加一
    return true;
};

template <class E>
bool SeqQueue<E>::getFront(E& x) const {
//若队列不空则函数返回该队列队头元素的值
    if (IsEmpty() == true) return false;    //队列空
    x = elements[front];                  //返回队头元素
    return true;
};
```

队列的链接表示 — 链式队列



- 队头在链头，队尾在链尾。
- 链式队列在进队时无队满问题，但有队空问题。
- 队空条件为 ***front == NULL***

链式队列类的定义

```
#include <iostream.h>
#include "Queue.h"
template <class E>
struct QueueNode {                          //队列结点类定义
private:
    E data;                                 //队列结点数据
    QueueNode<E> *link;                     //结点链指针
public:
    QueueNode(E x = 0, QueueNode<E>
      *next = NULL) : data(x), link(next) { }
};
```

```
template <class E>
class LinkedQueue {
private:
    QueueNode<E> *front, *rear;   //队头、队尾指针
public:
   LinkedQueue() : rear(NULL), front(NULL) { }
   ~LinkedQueue();
   bool EnQueue(E x);
   bool DeQueue(E& x);
   bool getFront(E& x);
   void makeEmpty();          //实现与~Queue()同
   bool IsEmpty() const { return front == NULL; }
};
```

```
template <class E>
LinkedQueue<E>::~LinkedQueue() {      //析构函数
    QueueNode<E> *p;
    while (front != NULL) {           //逐个结点释放
        p = front;  front = front->link;  delete p;
    }
};
```

```
template <class E>
bool LinkedQueue<E>::EnQueue(E x) {
//将新元素x插入到队列的队尾

    if (front == NULL)  {              //创建第一个结点
        front = rear = new QueueNode<E> (x);
        if (front == NULL) return false; }      //分配失败
    else {                                  //队列不空, 插入
        rear->link = new QueueNode<E> (x);
        if (rear->link == NULL) return false;
        rear = rear->link;
    }
    return true;
};
```

```
template <class E>
//如果队列不空，将队头结点从链式队列中删去
bool LinkedQueue<E>::DeQueue(E& x) {
    if (IsEmpty() == true) return false;        //判队空
    QueueNode<E> *p = front;
    x = front->data;  front = front->link;
    delete p;   return true;
};

template <class E>
bool LinkedQueue<E>::getFront(E& x) {
//若队列不空，则函数返回队头元素的值
    if (IsEmpty() == true) return false;
    x = front->data;  return true;
};
```

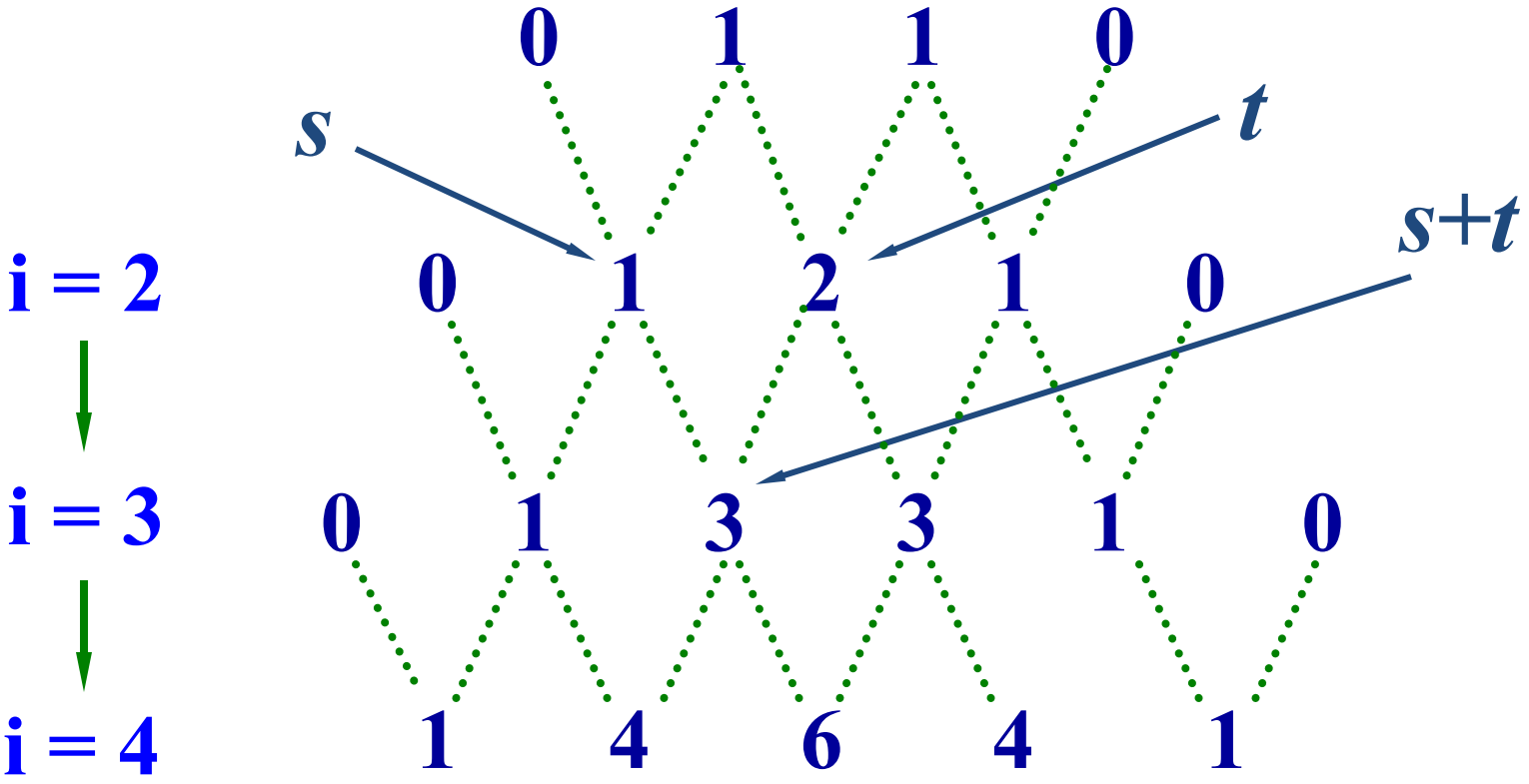
队列的应用:

逐行打印二项展开式 $(a+b)^i$ 的系数

杨辉三角形 (Pascal's triangle)

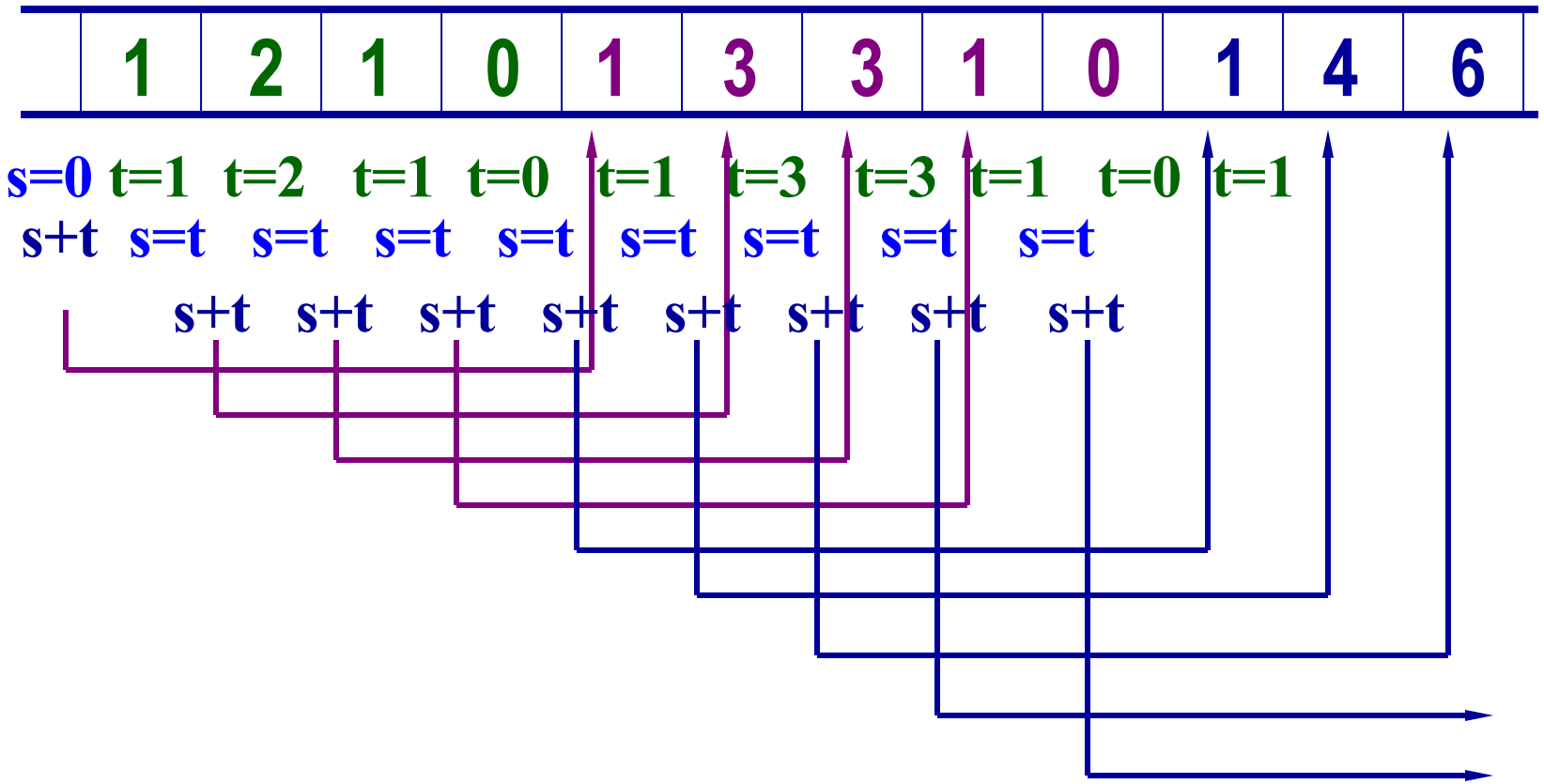
				1		1						$i=1$
			1		2		1					2
		1		3		3		1				3
	1		4		6		4		1			4
1		5		10		10		5		1		5
1		6		15		20		15		6	1	6

分析第 i 行元素与第 i+1行元素的关系



从前一行的数据可以计算下一行的数据

从第 i 行数据计算并存放第 i+1 行数据



利用队列打印二项展开式系数的算法

```
#include <stdio.h>
#include <iostream.h>
#include "queue.h"
void YANGHVI(int n) {
    Queue q(n+3);                    //队列初始化
    q.makeEmpty();
    q.EnQueue(1);  q.EnQueue(1);
    int s = 0, t;
```

```
    for (int i = 1; i <= n; i++) {          //逐行计算
        cout << endl;
        q.EnQueue(0);
        for (int j = 1; j <= i+2; j++) {    //下一行
            q.DeQueue(t);
            q.EnQueue(s + t);
            s = t;
            if (j != i+2) cout << s << ' ';
        }
    }
}
```

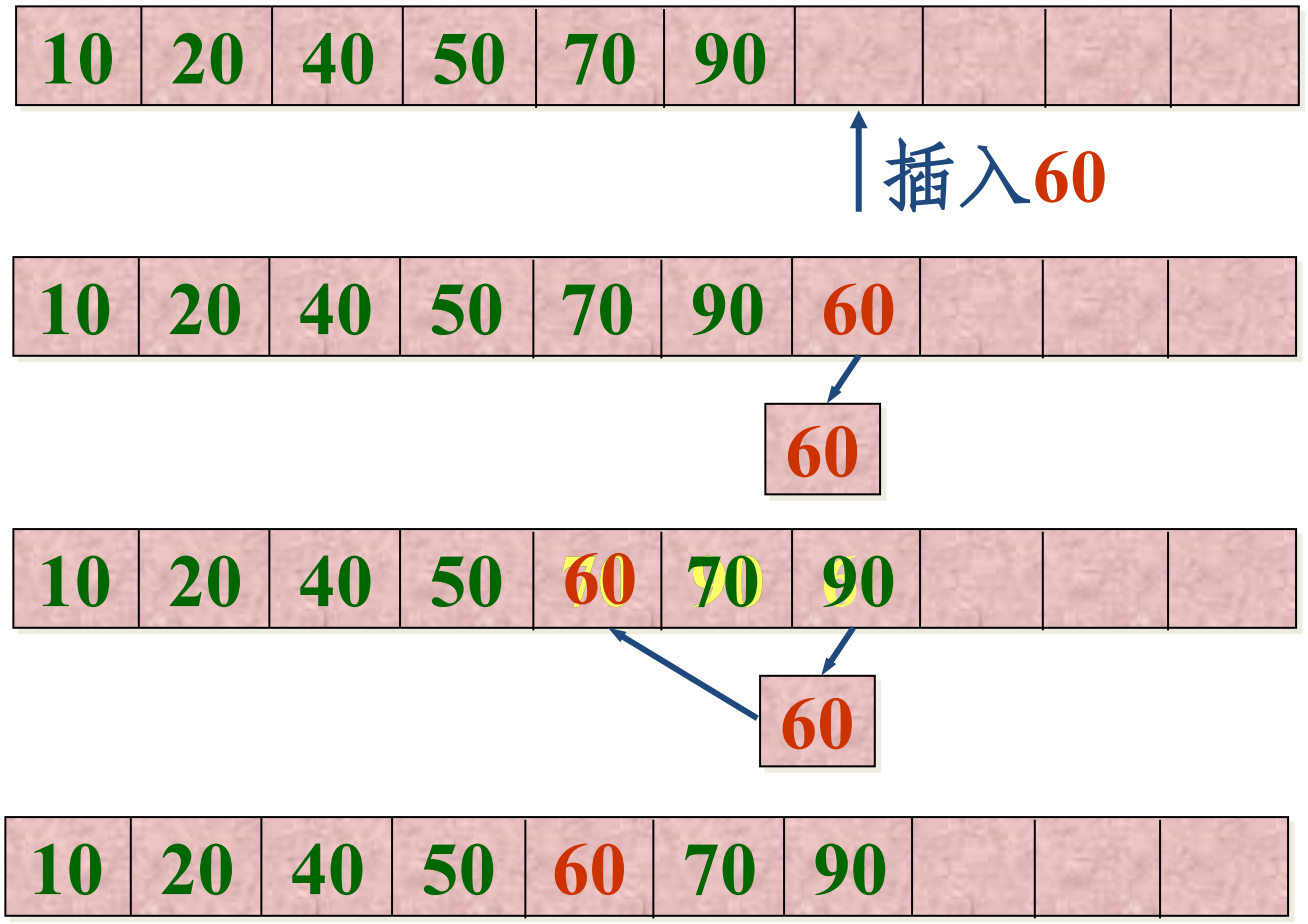
§3.4 优先级队列（priority queue）

任务编号	1	2	3	4	5
优先权	20	0	40	30	10
执行顺序	3	1	5	4	2

数字越小，优先权越高

- 优先级队列： 是不同于先进先出队列的另一种队列。每次从队列中取出的是具有最高优先权(优先级)的元素

- 优先权是根据问题而定的
- 出现相同的优先权的元素时，按*FIFO*的方式处理


10 20 40 50 70 90
插入60
10 20 40 50 70 90 60
60
10 20 40 50 60 70 90
60
10 20 40 50 60 70 90

优先级队列的类定义

```
#include <assert.h>
#include <iostream.h>
#include <stdlib.h>

template <class E>
class PQueue {
private:
    E *pqelements;              //存放数组
    int count;                  //队列元素计数
    int maxPQSize;              //最大元素个数
    void adjust();              //调整
```

```
public:
    PQueue(int sz = 50);
   ~PQueue() { delete [ ] pqelements; }
    bool Insert(E x);
    bool removeMin(E& x);
    bool getFront(E& x);
    void makeEmpty() { count = 0; }
    bool IsEmpty() const { return count == 0; }
    bool IsFull() const
          { return count == maxPQSize; }
    int Length() const { return count; }
};
```

优先级队列部分成员函数的实现

```
template <class E>
PQueue<E>::PQueue(int sz) {
    maxPQSize = sz;  count = 0;
    pqelements = new E[maxPQSize];
    assert (pqelements != NULL);
}

template <class E>
bool PQueue<E>::Insert(E x) {
    if (IsFull() == true) return false;   //判队满断言
    pqelements[count++] = x;              //插入
    adjust();   return true;
}
```

```
template <class E>
void PQueue<E>::adjust() {
    E temp = pqelements[count-1];
    //将最后元素暂存再从后向前找插入位置
    for (int j = count-2;  j >= 0;  j--)
        if (pqelements[j] <= temp) break;
        else pqelements[j+1] = pqelements[j];
    pqelements[j+1] = temp;
}
```

```
template <class E>
bool PQueue<E>::removeMin(E& x) {
    if (IsEmpty()) return false;
    x = pqelements[0];        //取出0号元素
    for (int i = 1; i < count; i++)
        pqelements[i-1] = pqelements[i];
        //从后向前逐个移动元素填补空位
    count--;
    return true;
}
```

```
template <class E>
bool PQueue<E>::getFront (E& x) {
    if (IsEmpty() == true) return false;
    x = pqelements[0];
    return true;
}
```